习题课-第6周

北京大学

目录

- 第二、三次上机题目
- 第二次书面作业

上机题目：字符串乘方

- 题目：对于字符串s，寻找最大的n，使得存在一个字符串a，让s=a^n。a^n表示字符串a重复n次。
  - 等同于寻找最小的循环节长度
  - 1. 暴力遍历即可。依次遍历len(s)的所有因数k，判断长为k的前缀是否构成循环节。
    - 对于每个k的判断，代价都是O(N)的（len(s)=N）
    - 因数的个数d(N)：渐进大于log N，但是小于任何多项式
  - 2. 利用s的next数组。
    - 最小的循环节 a，则有最长的公共前后缀 s – a
    - 只需要求出最长公共前后缀的长度k，然后判断 N-k 是否能够被N整除即可
    - 所求的 n = N / (N-k)。如果不整除，则 n = 1
    - 复杂度为O(N)

上机题目：堆栈基本操作

- 题目：依次读入序列元素1,2,...,n进栈，每进一个元素，机器可要求下一个元素进栈或弹栈，如此进行。给定一个输入序列，判断栈空时弹出的元素构成的序列是否可能等于给定的序列，如果是则输出栈的操作过程，否则输出“NO”。
  - 简单的模拟题目，下面是一种模拟方法
  - 遍历给定序列的元素。
    - 如果当前元素仍未进栈，就将压栈。在此之前，需要将所有较小的元素都压栈。然后将该元素出栈
    - 如果这个元素已经进栈，则它必须位于栈顶，才可以将其出栈。否则输出NO即可

上机题目：堆栈基本操作

- 第二次书面作业判断题第4题：若让元素 1，2，3 依次进栈，并允许在栈非空的任意时刻进行出栈操作，则在 3 个元素可能形成的 6 种全排列中，仅有一种序列不可能成为合法的出栈序列，即 3，1，2。（正确）
  - 首先出栈3，意味着必须连续压栈1，2，3。进而，2一定比1先出栈。
  - 对于更长的出栈序列，实际上该序列“合法”的充要条件就是不存在“3，1，2”式的结构
  - 严格来说：对于序列中的任意三个元素 a_i, a_j, a_k（$i<j<k$），如果 $a_i > a_k$，则必须满足 $a_j > a_k$
  - 例如长度为10的序列：4, 3, 6, 2, 5, 1, 8, 7, 10, 9

上机题目：最长上升子序列和

- 题目：给定一个长度为N的数字序列，求一个严格升序的子序列，使得子序列中元素之和最大。
  - 比如，对于序列(1, 7, 3, 5, 9, 4, 8)，有它的一些上升子序列，如(1, 7), (3, 4, 8)等等。
  - 这些子序列中序列和最大为18，为子序列(1, 3, 5, 9)的和。
  - 注意，最长的上升子序列的和不一定是最大的，比如序列(100, 1, 2, 3)的最大上升子序列和为100，而最长上升子序列为(1, 2, 3)
- 动态规划解法：
  - 行动一步：先任意挑选子序列的第一个元素，或最后一个元素
  - 形式上等价，不同的只在于后续子问题之间的依赖顺序。为了方便，我们考虑先挑选子序列的最后一个元素

上机题目：最长上升子序列和

- 剩下的子问题：给定序列尾元素，求子序列的最大和
  - 子问题与原问题形式并不同，但是注意到，原问题的解就是所有子问题的最优解的最大值。求解所有最大值即可
  - dp[i]定义为，尾元素为array[i]的升序子序列的最大和
- 再考虑子序列的倒数第二个元素array[j]的选取
  - 对于 $0 \le j < i$，如果array[i] > array[j]，array[j]才可能出现在dp[i]的解中
  - 状态转移方程：

$$dp[i] = \max\{dp[j]\} + array[i], \forall\, 0 \le j < i \text{ if } array[j] < array[i]$$

  - 考虑边界情况：最优解中只有一个元素：dp[i] = array[i]
- 时间复杂度：$O(N^2)$

上机题目：登山

- 题目：给定长度为N的序列表示N个景点的高度，要求找出最长的子序列长度，满足首先严格上升，然后严格下降
- 很自然地：考虑最高点位置，然后分别求升序前缀、降序后缀的长度
  - 两个问题形式上是对称的，将序列反转后这两个问题就互换了
  - 分解子问题的方式与前一个问题是相同的，只有状态转移方程不同：
  - dp[i]表示以array[i]结尾的升序子序列的最大长度
  - $dp[i] = \max\{dp[j]\}+1, \ \forall\, 0 \le j < i \text{ if } array[j] < array[i]$
  - 同样，如果最优解中只有一个元素，dp[i] = 1
- 时间复杂度为$O(N^2)$

上机题目：四则运算表达式求值

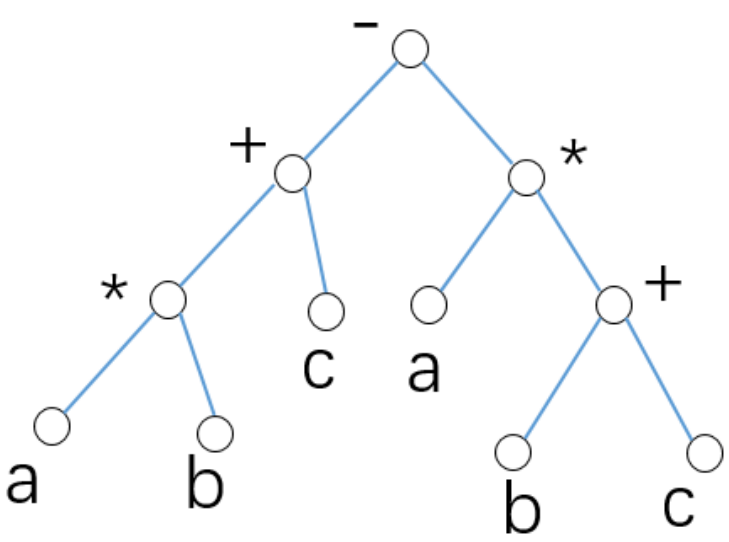
- 回忆：我们已经学习过的关于表达式求值的知识：
  - 中缀表达式、前缀表达式（波兰式）、后缀表达式（逆波兰式）
    - (A+B)*C， * + A B C，A B + C *
  - 利用栈将中缀表达式转换为逆波兰式（调度场算法，Dijkstra）
    - 遇到操作数，左括号，右括号，操作符时分别进行对应操作
  - 利用栈求值逆波兰式
    - 顺序扫描一遍即可
- 这道题目的意义在于希望同学们实现一遍这个流程
  - 尽量不要直接使用eval()函数

上机题目：四则运算表达式求值

- 能否利用波兰表达式完成中缀表达式的求值过程？
- 中缀表达式转换为波兰式：
  - 将中缀表达式逆序，并交换左括号与右括号
  - 求逆序中缀表达式的逆波兰式，再逆序就得到原中缀表达式的波兰式
- 用表达式二叉树来理解这一点
  - 逆波兰式：后序遍历（左，右，根）
  - 将原中缀式逆序，相当于交换了所有结点的左右子树
  - 求逆序中缀式的逆波兰式（右，左，根）
  - 再逆序，得到先序遍历序列（根，左，右）
- 求值波兰式（上机题目：波兰表达式）
  - 以同样的方式，利用栈逆序遍历一遍即可

表达式二叉树

- 前序（前缀）： - + * a b c * a + b c
- 中序 ： a * b + c - a * (b + c)
- 后序（后缀）：a b * c + a b c + * -



书面作业

- 判断题1：字符串是特殊的线性表，特殊性体现在其内容限制为字符。（正确）
  - 线性表：具有线性的逻辑结构。字符串的逻辑结构是线性的
  - 字符串的内容限制为字符：指字符串的每个元素类型都是字符类型
- 疑问：
  - “字符串的特殊性体现在基本操作通常以串的整体作为操作对象”
    - 与这道题目的论断并不矛盾
  - 字符串不可变，例如不能修改指定下标的元素
    - Python 中确实不能，但其他语言中（例如C语言）就可以修改
    - 这是字符串的特定实现的性质，而不是字符串本身的性质

书面作业：KMP算法

- 考虑字符串模式匹配问题，T=’abaabaabaabcc’，P=’abaabc’。给出模式串 P 的改进前的和改进后的 next 数组，并分别给出，使用改进前的 KMP 算法、改进后的 KMP 算法、以及朴素模式匹配算法，各需要进行多少趟的匹配。
- Next数组：Next[i] 表示 P[0: i] 的最长公共前后缀长度

i	**0**	**1**	**2**	**3**	**4**	**5**
P	a	b	a	a	b	c
P[0:i]最长公共前后缀长度	--	0	0	1	1	2
Next[i]	-1	0	0	1	1	2

书面作业：KMP算法

- 匹配过程

i	0	1	2	3	4	5
P	a	b	a	a	b	c
Next[i]	-1	0	0	1	1	2

T	a	b	a	a	b	a	a	b	a	a	b	c	c
第1趟	a	b	a	a	b	c							

- 在P[5] ≠ T[5] 处失配
- next[5]=2，下一步继续匹配 P[2]与T[5]

书面作业：KMP算法

- 匹配过程

i	0	1	2	3	4	5
P	a	b	a	a	b	c
Next[i]	-1	0	0	1	1	2

T	a	b	a	a	b	a	a	b	a	a	b	c	c
第1趟	a	b	a	a	b	c							
第2趟				(a)	(b)	a	a	b	c				

- 在P[5] ≠ T[8] 处失配
- next[5]=2，下一步继续匹配 P[2]与T[8]

书面作业：KMP算法

- 匹配过程

i	0	1	2	3	4	5
P	a	b	a	a	b	c
Next[i]	-1	0	0	1	1	2

T	a	b	a	a	b	a	a	b	a	a	b	c	c
第1趟	a	b	a	a	b	c							
第2趟				(a)	(b)	a	a	b	c				
第3趟							(a)	(b)	a	a	b	c	

书面作业：KMP算法

- Next数组的改进：如果next[i] = k，且P[i] = P[k]
  - 例如 next[2] = 0, 且P[2]=P[0]。以及next[4]=1, P[4]=P[1]
- 那么在P[i]≠T[j]处失配时，按照改进前的next数组，继续比较P[k]与T[j]也还是会失配。改进前的KMP算法会继续查找next[k]进行比较
  - 例如：如果在P[4]处失配，按照next数组，会从P[1]开始比较
  - 由于P[1]=P[4]，P[1]也会失配。再依据next[1]=0，从P[0]处继续比较
  - 将next[4]直接设置成next[1]=0，在匹配时就避免了上述过程

i	**0**	**1**	**2**	**3**	**4**	**5**
P	a	b	a	a	b	c
Next[i]	-1	0	0	1	1	2

书面作业：KMP算法

- 优化前的Next数组

i	0	1	2	3	4	5
P	a	b	a	a	b	c
Next[i]	-1	0	0	1	1	2

- 优化后的Next数组

i	0	1	2	3	4	5
P	a	b	a	a	b	c
Next[i]	-1	0	-1	1	0	2

- 在这个例子里，Next数组的改进没有影响到匹配过程。匹配趟数都为3趟。

书面作业：KMP算法

- 朴素算法的匹配过程：共7趟

T	**a**	**b**	**a**	**a**	**b**	**a**	**a**	**b**	**a**	**a**	**b**	**c**	**c**
第1趟	a	b	a	a	b	c							
第2趟		a											
第3趟			a	b									
第4趟				a	b	a	a	b	c				
第5趟					a								
第6趟						a	b						
第7趟							a	b	a	a	b	c	

书面作业：KMP算法

- 区分匹配过程的“趟数”与“次数”
- 一趟匹配：在某个位置s上，尝试匹配 T[s: s+m] = P，m为P的长度。
  - 该问题中，改进前后的KMP算法都进行了3趟匹配，朴素算法进行了7趟匹配
- 一次匹配：比较两个字符是否相等，是衡量匹配算法时间复杂度的基本操作
  - 该问题中，改进前后的KMP算法都进行了14次匹配，朴素算法进行了24次匹配

书面作业：编辑距离

- 题目：若 S, T 是两个字符串，对 S 进行如下三种操作：修改一个字符、在任意位置增加一个字符、删除任意一个字符，以使 S 转换为 T；则称所需要操作的最少数量为 S 与 T 之间的编辑距离。
- L(i, j)表示S[:i]与T[:j]的编辑距离
- 考虑S串中最终与T[-1]匹配的字符的几种情况：
  1. S[-1] == T[-1]，此时L(i, j) = L(i-1, j-1)；
  2. S[-1]经修改与T[-1]匹配，此时L(i, j) = 1+L(i-1, j-1)；
  3. 在S[-1]之后新增了字符，使之与T[-1]匹配，此时L(i, j) = 1+L(i, j-1)
  4. S[-1]被删除，使得之前的字符与T[-1]匹配。此时L(i, j) = 1+L(i-1, j)

书面作业：编辑距离

- 状态转移方程：

$$L(i,j)=\begin{cases}L(i-1,j-1), & \text{if } S[i-1]=T[j-1]\\ 1+\min\{L(i-1,j-1),\ L(i,j-1),\ L(i-1,j)\}, & \text{otherwise}\end{cases}$$

- L(i, j)依赖于L(i-1, j-1), L(i, j-1), L(i-1, j)
- 算法设计：
  - 设置二维数组dp。dp[i][j]表示L(i, j)的解。
  - 将dp[0][j]初始化为j，将dp[i][0]初始化为i
  - 顺序循环所有 i，并嵌套顺序循环所有 j，计算L(i, j)。
  - 返回dp[m][n]
- 时间复杂度为O(m*n)

书面作业：鸡蛋的硬度

- 题目：
  - M层楼，N个完全相同的鸡蛋
  - 如果L楼没有碎，L+1楼碎，鸡蛋的硬度就定义为M。硬度最大为M
  - 要求设计一个方案，使得最坏情况下的测试次数最少
- 理解“测试方案”与“最坏情况”
  - 鸡蛋的硬度是未知的，一次试验后，可能碎，可能不碎
  - 测试方案应该给出，在任何情况下应当如何做
  - 例如一个测试方案是：先在1楼测试。如果碎了，测试结束；如果没碎，则在2楼测试。如果碎了，测试结束；如果没碎……
  - 该方案下最坏情况就是，鸡蛋的硬度为M。最坏情况下需要测试M次
  - 只有一个可用的鸡蛋时，这是唯一可行的方案
    - 第(1)问就应当采取该方案，最坏情况的测试次数为k

书面作业：鸡蛋的硬度

- 动态规划：
  - 行动一步，第一次测试发生在k楼
  - 如果鸡蛋破碎了，对应子问题 S(k-1, N-1)；
  - 如果鸡蛋未破碎，对应子问题 S(M-k, N)
- 状态转移方程：

$$S(i, j) = 1+ \min\{\max\{S(k-1, j-1), S(i-k, j)\}\},\quad 1\le k\le i。$$

  - S(i, j) 依赖于所有 S(k-1, j-1), S(i-k, j)
  - 顺序遍历所有i, j 即可
- 时间复杂度为$O(M*N^2)$
  - 算法设计完成之后，不要忘记分析时间复杂度

书面作业：鸡蛋的硬度

- 进阶解法：考虑问题的对偶问题
  - 使用S次测试机会和N个鸡蛋，最多可以在多高的大楼中，在最坏情况下也能保证确定鸡蛋的硬度。
  - 例如，如果N=1，使用S次机会最多能在S层楼中，在最坏情况下保证测试成功。S+1层楼的最坏情况就需要S+1次测试机会。
- 仍然使用动态规划求解对偶问题
  - dp[i][j]表示使用 i 次测试机会，j个鸡蛋，最大可行的楼层数
  - 存在一个方案P，能够满足在dp[i][j]层楼中测试成功
  - 这个P方案的第一次测试发生在楼层k
    - 如果碎了，鸡蛋的硬度位于[0, k-1]中。k-1 = dp[i-1][j-1]
    - 如果没碎，鸡蛋的硬度位于[k, dp[i][j]]中，dp[i][j] – k = dp[i-1][j]

书面作业：鸡蛋的硬度

- 状态转移方程：

$$dp[i][j] = dp[i-1][j-1] + dp[i-1][j] + 1$$

- dp[i][j]仅依赖于dp[i-1][j-1], dp[i-1][j]，顺序遍历i, j 即可

- 由对偶问题转化到原问题：
  - 不断增加测试机会数 i，求出dp[i][N]，N为可用的鸡蛋数量
  - 第一次达到dp[i][N] ≥ M时，i就是原问题的答案
- 时间复杂度为O(tN)，t为原问题的答案规模